.NET Multi-platform App UI

David Ortinau | @davidortinau

January 28, 2021



David Ortinau

A Personal Timeline

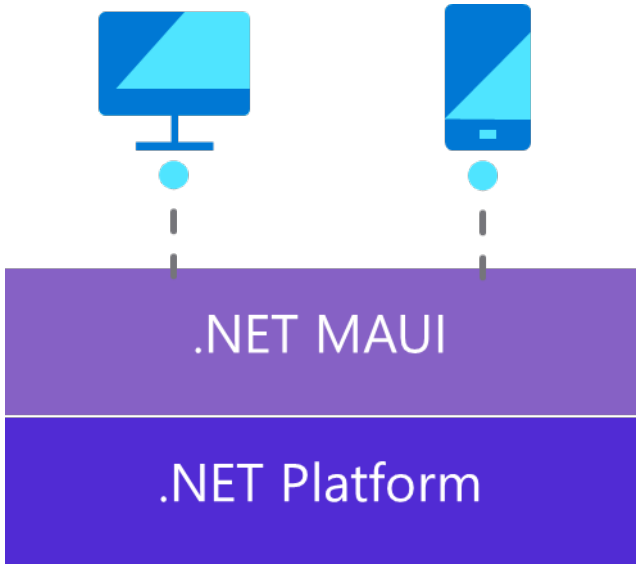
‘96-99	HTML/JS, media streaming, ASP, SQL
’99-02	Flash, ASP, SQL
’02-07	.NET, SQL, Flash, Flex
’07-10	Rails, Django, PHP, Flash, Flex, Silverlight, iOS, Android
’11-13	MonoTouch, MonoDroid, iOS, Android, Flash, Flex
‘14-17	Xamarin, Flash, Flex
’17-21	Xamarin
2021	.NET MAUI



.NET MAUI is the evolution of Xamarin.Forms extended from mobile to desktop scenarios with UI controls rebuilt from the ground up for performance and extensibility.

.NET Multi-platform App UI (MAUI)

- Xamarin integrated into .NET
  - github.com/dotnet/maui
  - github.com/dotnet/android
  - github.com/dotnet/macios
- Common .NET experiences
  - CLI
  - Visual Studio Code
- Android, iOS, macOS, and Windows
- Support for modern app models like Blazor and Comet



EXPERIMENTAL

Goals of .NET MAUI

Quality

12 months of Xamarin quality releases

Reorganized project structure

Standardized class naming and structure

Performance

Cold start <= 1000ms

Responsive UI rendering, animations, and transitions

Reduced view nesting

Design

Simple and elegant default UI theme

Easy control extensibility

Quick cross-platform templating

Platforms

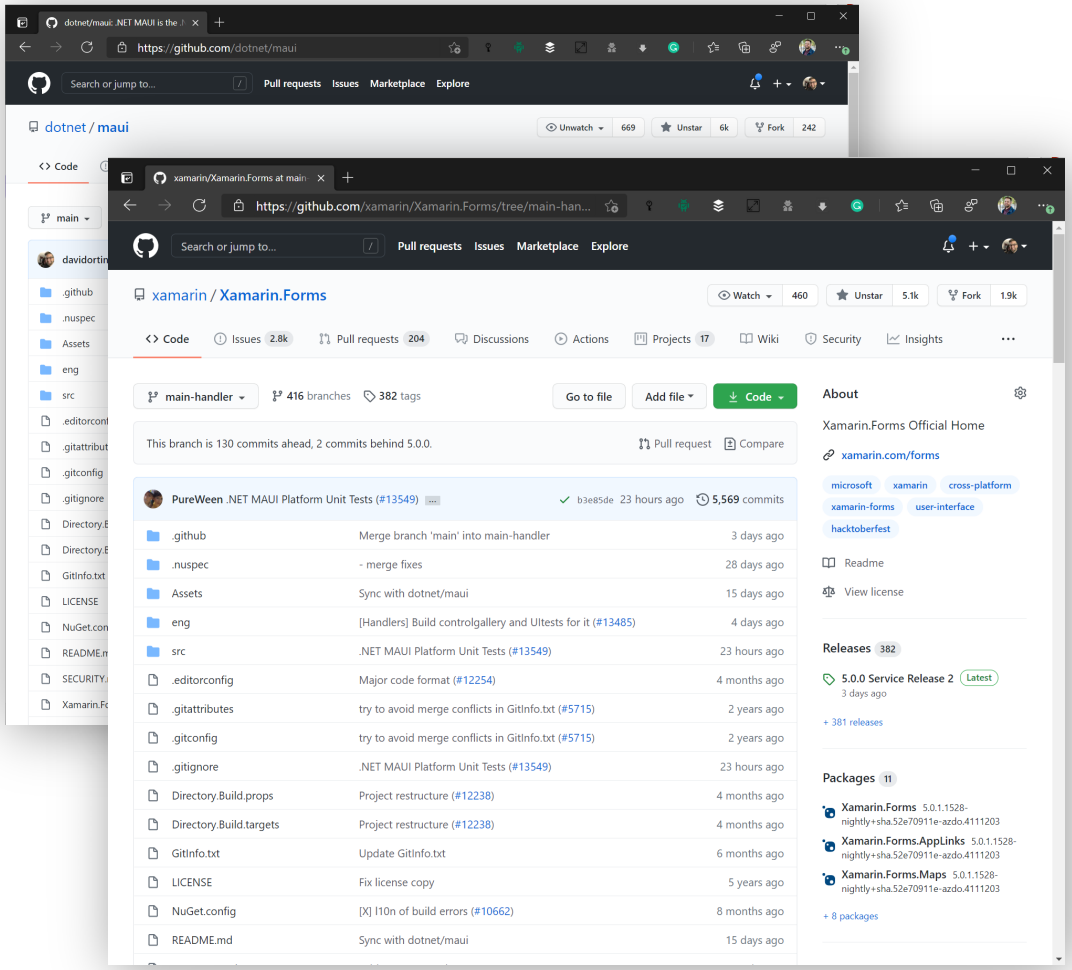
Android

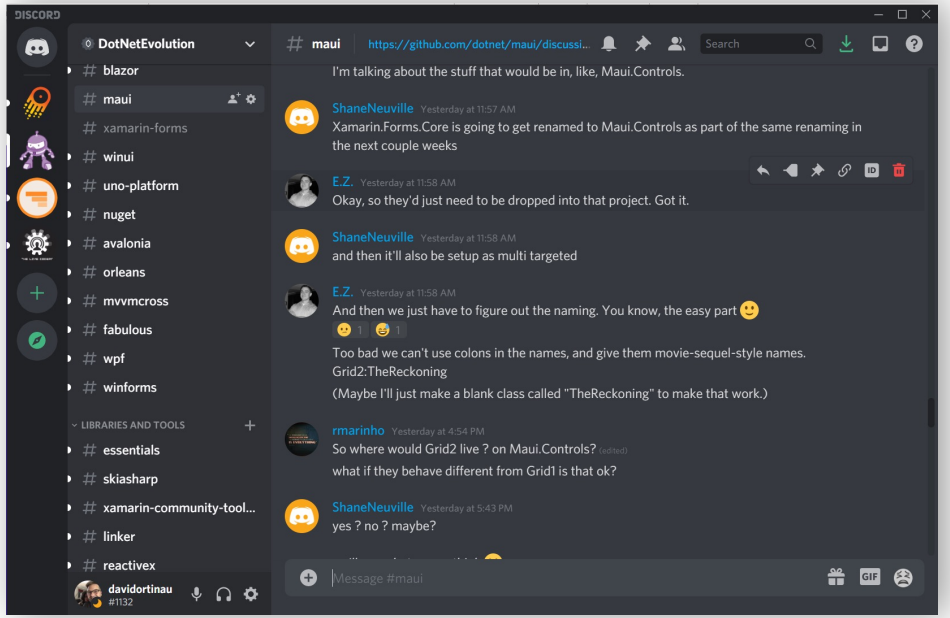
iOS

macOS

Windows

Where is this happening?





https://aka.ms/discord-maui

Open | Closed

- **.NET appeals to net-new developers, students, and new technologists** Theme
  Proposed for .NET 6.0 dotnet/core#5465 opened one month ago
- **.NET has a great client app development experience** Theme
  Proposed for .NET 6.0 dotnet/core#5423 opened one month ago
  - **Blazor developers can target and deploy their applications to desktop platforms, allowing them to run natively** Epic Priority:0 affected-medium area-blazor enhancement severity-blocking
    Proposed for .NET 6.0 dotnet/aspnetcore#27217 opened one month ago Next sprint planning
  - **Desktop developers can package, distribute, release, and update their applications to multiple desktop platforms and architectures** Cost:XL Epic Priority:0
    Proposed for .NET 6.0 dotnet/core#5462 opened one month ago Future rconard
  - **Xamarin developers can upgrade to and use the latest .NET SDKs in their existing applications** Epic Priority:0 Theme: great client app dev XP
    Proposed for .NET 6.0 dotnet/xamarin#2 opened one month ago
    - **.NET developers can run iOS and Android apps on emulators and devices in .NET 6** Cost:XL Priority:0 User Story area-Meta
      Proposed for .NET 6.0 dotnet/runtime#44654 opened 19 days ago steveisok
    - **NuGet packages are updated and available to target .NET 6** Cost:M Priority:1 Theme: great client app dev XP User Story
      Proposed for .NET 6.0 dotnet/xamarin#15 opened 19 days ago
    - **Xamarin developers can acquire and manage mobile .NET SDKs** Cost:M Priority:0 Theme: great client app dev XP User Story
      Proposed for .NET 6.0 dotnet/xamarin#4 opened one month ago
    - **Xamarin developers can leverage a CLI try-convert tool to move basic Xamarin projects to the new project system** Cost:M Priority:1 Theme: great client app dev XP User Story
      Proposed for .NET 6.0 dotnet/xamarin#9 opened one month ago
    - **Xamarin developers can target all supported platforms from a single project, and share resources across them** Cost:XL Priority:0 User Story
      Proposed for .NET 6.0 dotnet/xamarin#11 opened one month ago
    - **Xamarin developers can target Windows with .NET 6** Cost:XL Priority:2 User Story
      Proposed for .NET 6.0 dotnet/xamarin#12 opened one month ago
    - **Xamarin developers can use refactored Xamarin.Core controls for improved performance** Cost:XL Priority:0 User Story
      Proposed for .NET 6.0 dotnet/xamarin#10 opened one month ago
    - **Xamarin developers will experience improved app startup with .NET 6** Cost:L Priority:1 Theme: great client app dev XP User Story
      In Progress for .NET 6.0 dotnet/xamarin#8 opened one month ago
    - **Xamarin developers will share more common developer experiences with .NET developers** Cost:XL Priority:0 Theme: great client app dev XP User Story
      In Progress for .NET 6.0 dotnet/xamarin#7 opened one month ago
    - **Xamarin developers will still be able to develop their legacy projects with Visual Studio** Cost:S Priority:0 Theme: great client app dev XP User Story
      In Progress for .NET 6.0 dotnet/xamarin#6 opened one month ago
    - **Xamarin is integrated with common .NET 6 targets and build commands** Cost:XL Priority:0 Theme: great client app dev XP User Story
      In Progress for .NET 6.0 dotnet/xamarin#16 opened 19 days ago
    - **Xamarin/Blazor developers can target macOS with .NET 6** Cost:L Priority:1 User Story area-Meta
      Proposed for .NET 6.0 dotnet/runtime#44736 opened 19 days ago marek-safar

.NET MAUI Timeline

- Preview 1 - Shipped
  - What's New
    - Runtime and BCL
  - Who should use this release?
    - Contributors
- Preview 2 - Shipped
  - What's New
    - .NET MAUI views, layouts, everything that's been merged
    - Android & iOS will be most complete
  - Who should use this release?
    - Contributors

.NET MAUI Timeline

- Preview 3 & 4
  - What's New
    - .NET MAUI will have most controls, layouts, pages, and services
    - Mac Catalyst & WinUI 3
  - Who should use this release?
    - You!
  - What should you do with this release?
    - Build new apps and kick the tires
    - Open issues and join discussion on Discord

.NET MAUI Timeline

- Preview 5 – Release Candidate
  - What's New
    - .NET MAUI fills the remaining gaps, responds to feedback
  - Who should use this release?
    - Everyone!
  - What should you do with this release?
    - Build new apps and kick the tires
    - Migrate existing Xamarin.Forms apps
    - Open issues and join discussion on Discord

Status Update

January 28, 2021

.NET 6 Build and Runtime

- On track!
  - MAUI is building on a branch with .NET 6 w/ tooling workarounds
  - Awaiting P1 and tooling updates to work against .NET 6 full-time
  - Collaborating with CoreCLR and Mono teams on macOS runtime
  - In Progress
    - Android and iOS ICU
    - Android Crypto
    - Android TLS
- Owners: David Ortinau, Jon Dick

.NET 6 Hot Reload

- On track!
  - Multi-team effort underway establishing the infrastructure for all workloads and platforms
  - Appearing first on Windows for desktop
- Owners: Dmitry Lyalin and friends

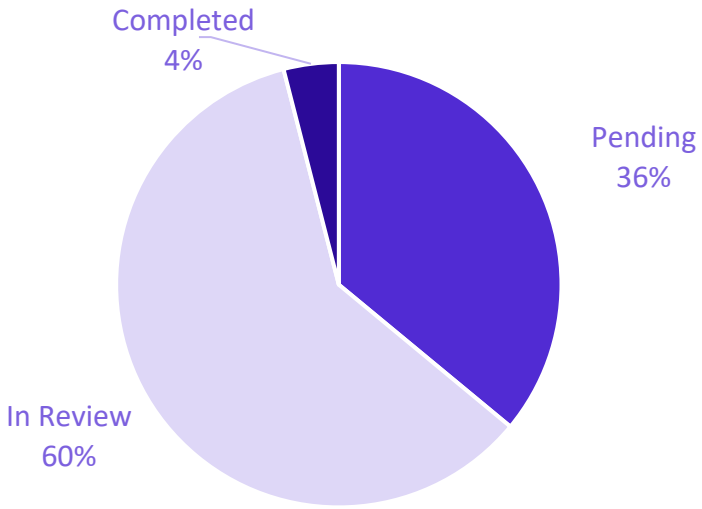
Single Project

- On track!
  - Working through developer experience for Fonts, Images, and App Icons
  - Multi-targeting
- Owners: Jake Kirsch, Jon Dick

Mobile SDKs

- Android: On track!
  - Emulator & Device deploy and run are working
  - AOT and Startup Tracing are in progress
- iOS: On Track
  - Simulator is working
  - In Progress
    - Device
    - AOT
    - Catalyst
    - Mac Build Agent

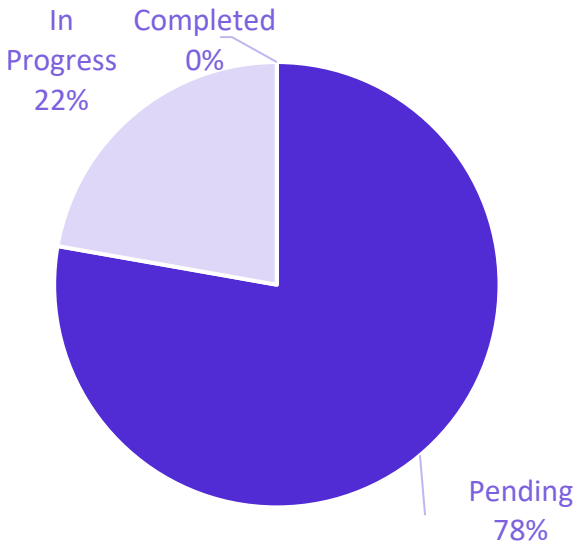
Views



Handler	Status
ActivityIndicator	In Progress
BoxView	In Progress
Button	In Progress
CheckBox	In Progress
DatePicker	In Progress
Entry	In Progress
Frame	In Progress
Image	In Progress
Label	In Progress
Picker	In Progress
ProgressBar	In Progress
SearchBar	In Progress
Shapes	In Progress
Slider	Completed
Switch	In Progress
TimePicker	In Progress

Layouts

Handler	Status
Grid	In Progress
StackLayout	In Progress

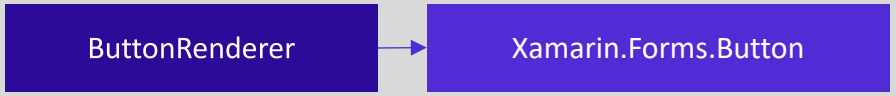


Evolving the Architecture

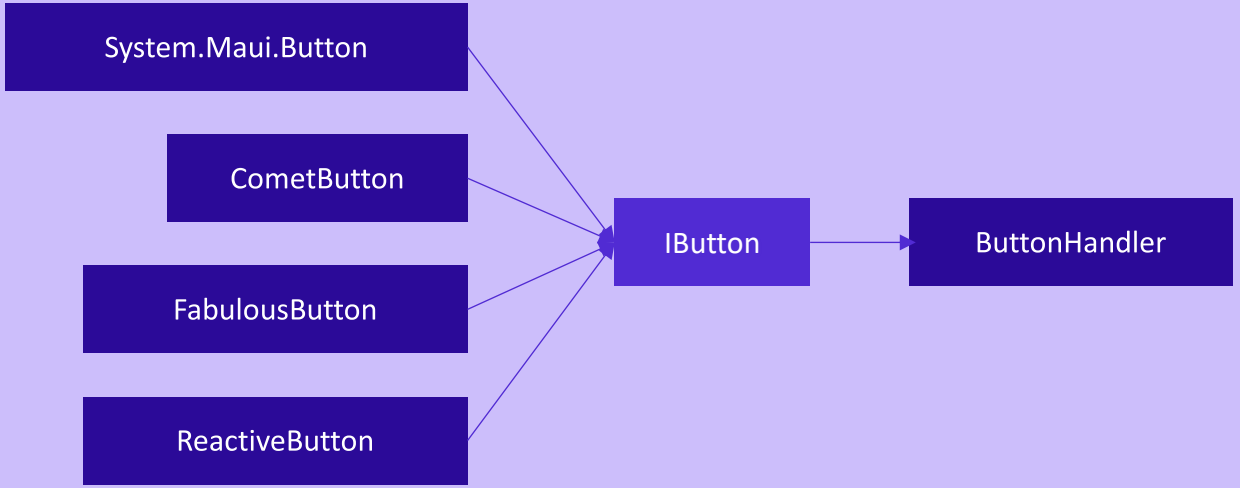
Renderers are tightly coupled to Xamarin.Forms and Xamarin.Forms components

Handlers are unwrapped, decoupled, portable, and reusable

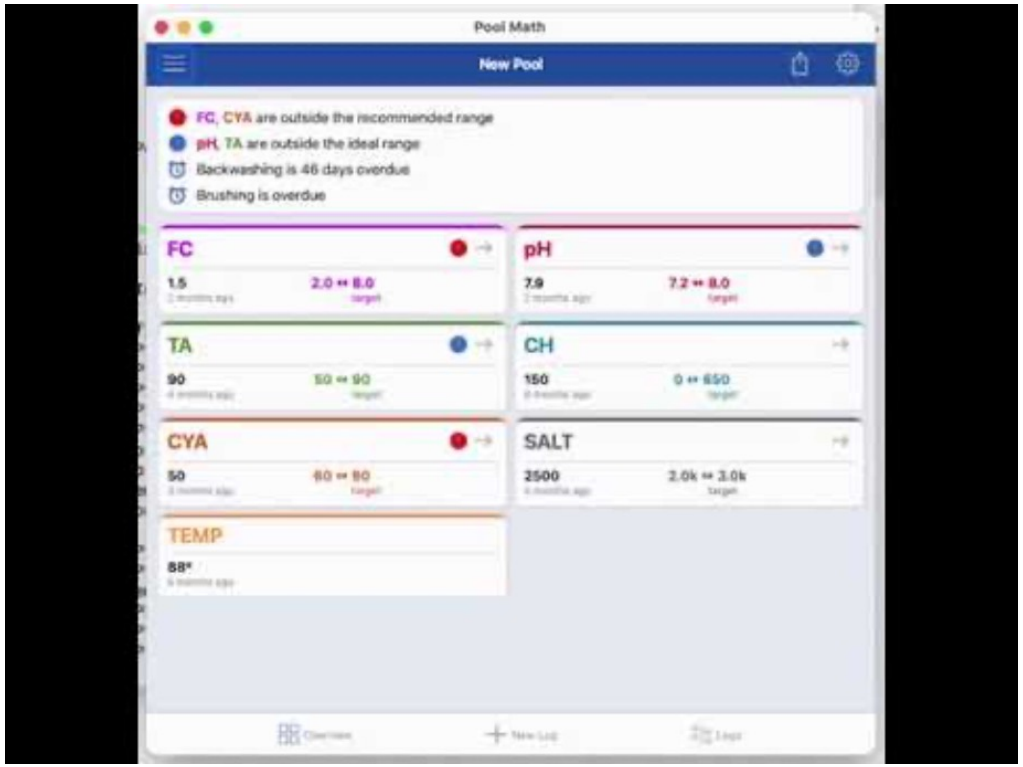
OLD



NEW

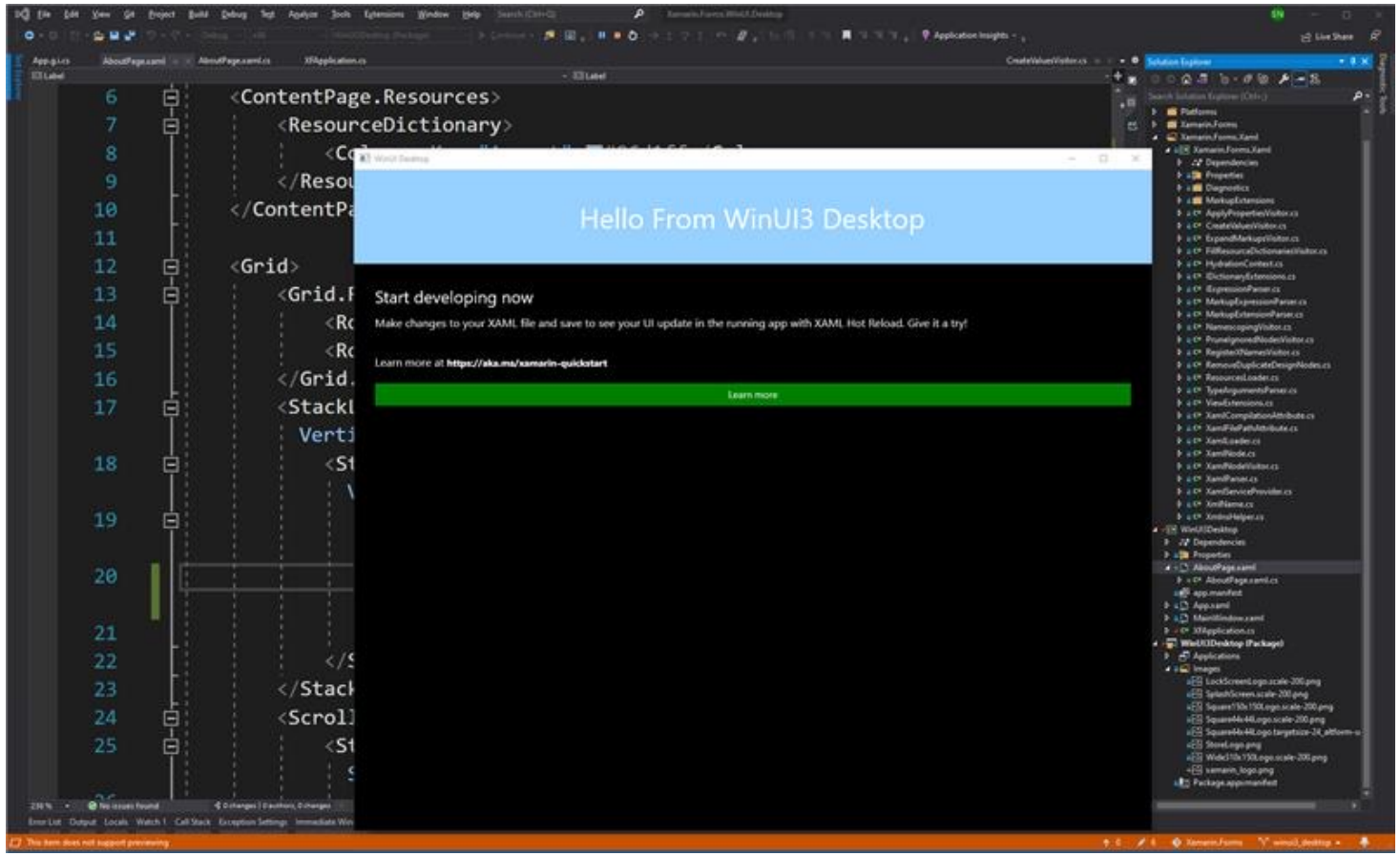


Mac Catalyst



Jonathan Dick – Pool Math
Thanks Frank! https://praeclarum.org/2020/11/10/catalyst.html

WinUI 3



https://github.com/xamarin/Xamarin.Forms/tree/winui3_desktop

OSS Library Maintainer - Monthly Sync

- 1st Tuesday of the Month @ Noon PT / 3 ET
- Teams Call to deep dive on arch and ask ?s
- Current representation from:
  - Prism
  - MvvmCross
  - FreshMvvm
  - ReactiveUI
  - Shiny
  - Xamarin Community Toolkit
  - Mobile Build Tools

To join:

david.ortinau@microsoft.com

Thanks for joining!
Questions?

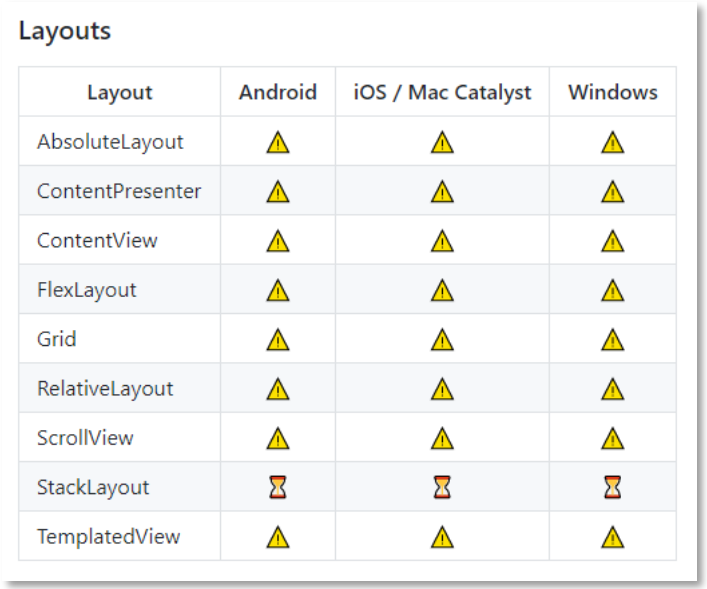


@davidortinau



Standup

- Porting renderers to mapper/handler architecture
- Porting layouts: StackLayout



Layouts

Layout	Android	iOS / Mac Catalyst	Windows
AbsoluteLayout	⚠	⚠	⚠
ContentPresenter	⚠	⚠	⚠
ContentView	⚠	⚠	⚠
FlexLayout	⚠	⚠	⚠
Grid	⚠	⚠	⚠
RelativeLayout	⚠	⚠	⚠
ScrollView	⚠	⚠	⚠
StackLayout	⌛	⌛	⌛
TemplatedView	⚠	⚠	⚠

Standup

- Porting renderers to mapper/handler architecture
- Porting layouts: StackLayout
- Control registrar – no reflection or assembly scan by default

```
RegistrarHandlers.Handlers.Register<Xamarin.Forms.Label, LabelHandler>();

#if MONOANDROID
RegistrarHandlers.Handlers.Register<Xamarin.Forms.Label,
Xamarin.Forms.Platform.Android.LabelRenderer>();
RegistrarHandlers.Handlers.Register<Xamarin.Forms.Button,
Xamarin.Forms.Platform.Android.AppCompat.ButtonRenderer>();
#endif
```

Standup

- Porting renderers to mapper/handler architecture
- Porting layouts: StackLayout
- Control registrar – no reflection or assembly scan by default
- Single Project

```
public class Startup
{
    // This method gets called by the runtime. Use this method to add services to
the container.
    public void ConfigureServices(IServiceCollection services)
    {
        services.AddFormsHandlers()
            .RegisterHandler<Button, MyButtonHandler>();
    }
```

```
public class Startup
{
    …

    // This method gets called by the runtime to configure the HTTP request pipeline.
    public void Configure(IApplicationBuilder app, IAppHostEnvironment env)
    {
        // Forms App type can be specified
        app.UseForms<App>();

        // Register Lifecycle handlers
        app.RegisterLifecycleHandler<MyCrossPlatLifecycleHandler>();

#if ANDROID
            app.RegisterAndroidLifecycleHandler<MyAndroidLifecycleHandler();
#elif IOS
            app.RegisteriOSLifecycleHandler<MyiOSLifecycleHandler();
#endif

        // Third party plugins can add their own IApplicationBuilder extensions
        app.UseShiny();
    }
}
```

Standup

- Porting renderers to mapper/handler architecture
- Porting layouts: StackLayout
- Control registrar – no reflection or assembly scan by default
- Single Project
  - Build task spike for automatic font embedding
  - Microsoft.Extensions.* Host Builder spike
- .NET for Android and iOS (xamarin/net6-samples)

.NET MAUI SDK Timeline

Q4 2020 – Q1 2021

Feature	Description	Status
Namespace changes to System.Maui	Xamarin.Forms will become System.Maui	Deferred
Handlers: Containers, Layouts, Views	Hello performance and extensibility	In Progress
Remove Obsoletes	Housekeeping	In Progress
Host Builder	.NET unified experiences	In Progress
Remove Data Pages	You use it? Probably not. Why is it still there?	Completed
Remove Themes	See Data Pages	Completed
Startup registry	No more reflection	Completed

.NET MAUI SDK Timeline

Q2 2021 – Q3 2021

Feature	Description	Status
Intro Cross-platform Lifecycle		Committed
Move “AndExpand” layout option	Belongs only on StackLayouts	Committed
Width/Height	Deprecate “Request” props, update behavior	Committed
Merge Essentials	Remove overlapping APIs	Committed
Improved native images	GlideX	Committed
Housekeeping	IAnimatable, Behavior/Trigger inheritance	Committed
macOS	Catalyst	Committed
Windows	WinUI 3 pending GA	Committed

Releases in sync with .NET 6 Previews
November 2021 GA

	Xamarin.Forms 5	.NET MAUI
Platforms		
Android	API 19+	API 21+
iOS	9 – 15	10+
Linux (GTK#)	Community	Community
macOS	Community	Microsoft
Tizen	Samsung	Samsung
Windows	UWP Microsoft WPF Community	WinUI 3 Microsoft WPF Community
Features		
Renderers	Tightly coupled to BindableObject	Loosely coupled, no Core deps
App Models	MVVM / RxUI	MVVM / RxUI (MVU experimental)
Single Project	No	Yes
Multi-Targeting	No	Yes
Multi-Window	No	Yes
Pixel Drawn Controls	No	Yes
Misc		
.NET	X.iOS, X.Android, UWP, …	.NET 6 +
Project System	Franken-Proj	SDK Style
dotnet CLI	No	Yes
Tools		
Visual Studio 2019	Yes	Yes
Visual Studio 2019 for Mac	Yes	Yes
Visual Studio Code	No	Experimental

Platform Code in .NET MAUI

Remove confusing APIs

```
using Xamarin.Forms.PlatformConfiguration;
using Xamarin.Forms.PlatformConfiguration.iOSSpecific;
boxView.On<iOS>().UseBlurEffect(BlurEffectStyle.ExtraLight);

[assembly: ExportRenderer (typeof(MyEntry),
typeof(MyEntryRenderer))]
namespace CustomRenderer.iOS {
        public class MyEntryRenderer : EntryRenderer
```

Easily reach the native platform from cross-platform code

```
#if MONOANDROID
ViewHandler.ViewMapper[nameof(IView.BackgroundColor)] = (handler, view) =>
     (handler.NativeView as AView).SetBackgroundColor(Xamarin.Forms.Color.Purple.ToNativ


var textView = crossPlatformLabel.Handler.NativeView as TextView;
#endif
```

Installing .NET MAUI

From the beginning

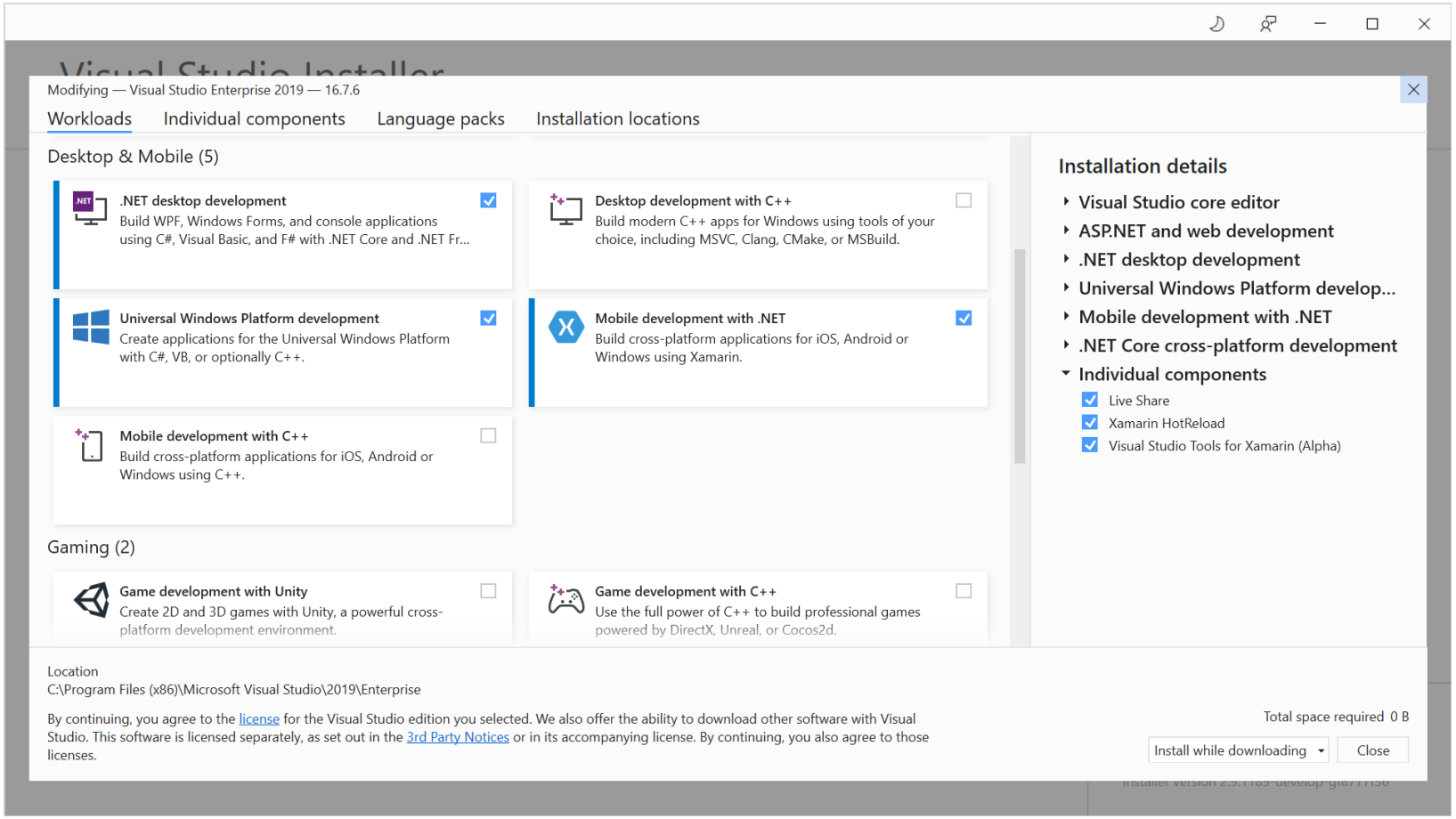
.NET Installer

- Optional workload packs (design proposal)
  - Android, iOS, macOS, WPF, etc.
  - .NET MAUI

- SDKs are decoupled from tooling
- Side by side SDK support for OS releases
- Ship with .NET
- Hosted as NuGet SDKs

```
dotnet install maui
```

Visual Studio Installer



Migrating to .NET MAUI

When you say “easy” what exactly do you think is easy?!?!

Moving to SDK Style projects

- try-convert
  - Tool will make a best effort to convert your csproj to SDK Style
  - Update target framework monikers
  - Run from CLI
  - Investigating IDE integration

Targeting .NET MAUI



- Namespaces
  - try-convert or similar will be implemented to move to System.Maui
- Custom Renderers
  - Recommendations
    - Remove those you no longer need
    - Replace the rest with handlers
  - Use built-in adapter shim and Xamarin.Forms.Legacy
- Build and fix
  - Deprecated APIs
  - Fixed APIs

Xamarin.Forms.Legacy

This is a (yet to exist) NuGet package where we can put things that you'll need to maintain compatibility for Xamarin.Forms "things" until you can complete any migration to .NET MAUI.

- Compatibility package
- Renderers
- Obsoleted

Third-Party Ecosystem

- Control Vendors & Third-Party Libraries
  - Recommendations
    - Update to .NET MAUI optimized packages
    - Recompile and target .NET 6 TFMs
  - Use adapter-shim and Xamarin.Forms.Legacy

- Framework Libraries (Prism, FreshMvvm, MvvmCross, RxUI...)
  - Recommendations
    - Update to .NET MAUI optimized packages

- We host a monthly sync call. Open invitation. 

How long is the transition period?

- November 2020
  - Announcements and Plans begin
  - Begin building migration help and producing guidance
- November 2021
  - Xamarin.Forms 5.x with latest iOS and Android support
  - .NET MAUI GA
- November 2022
  - Xamarin.Forms end of support
  - .NET MAUI vNext

Preparing for .NET MAUI

What should you do today to make for an easy migration

DO

Use AppCompat on Android

DO

Use Fast Renderers

DO NOT

Freak out



DO

Remove unnecessary custom renderers

DO

Purchase a Hawaiian Shirt



DO NOT

Use obsoleted APIs

DO NOT

Use DataPages or Themes NuGets

DO

Use .NET Standard Libraries

Evergreen .NET MAUI

What’s my new experience in .NET MAUI?

Building UI in .NET 6

All the XAML and MVVM you know and love

```xaml
<StackLayout>
    <Label Text="Welcome to Maui!" />

    <Button Text="{Binding Text}"
            Clicked="{Binding ClickCommand}" />
</StackLayout>
```

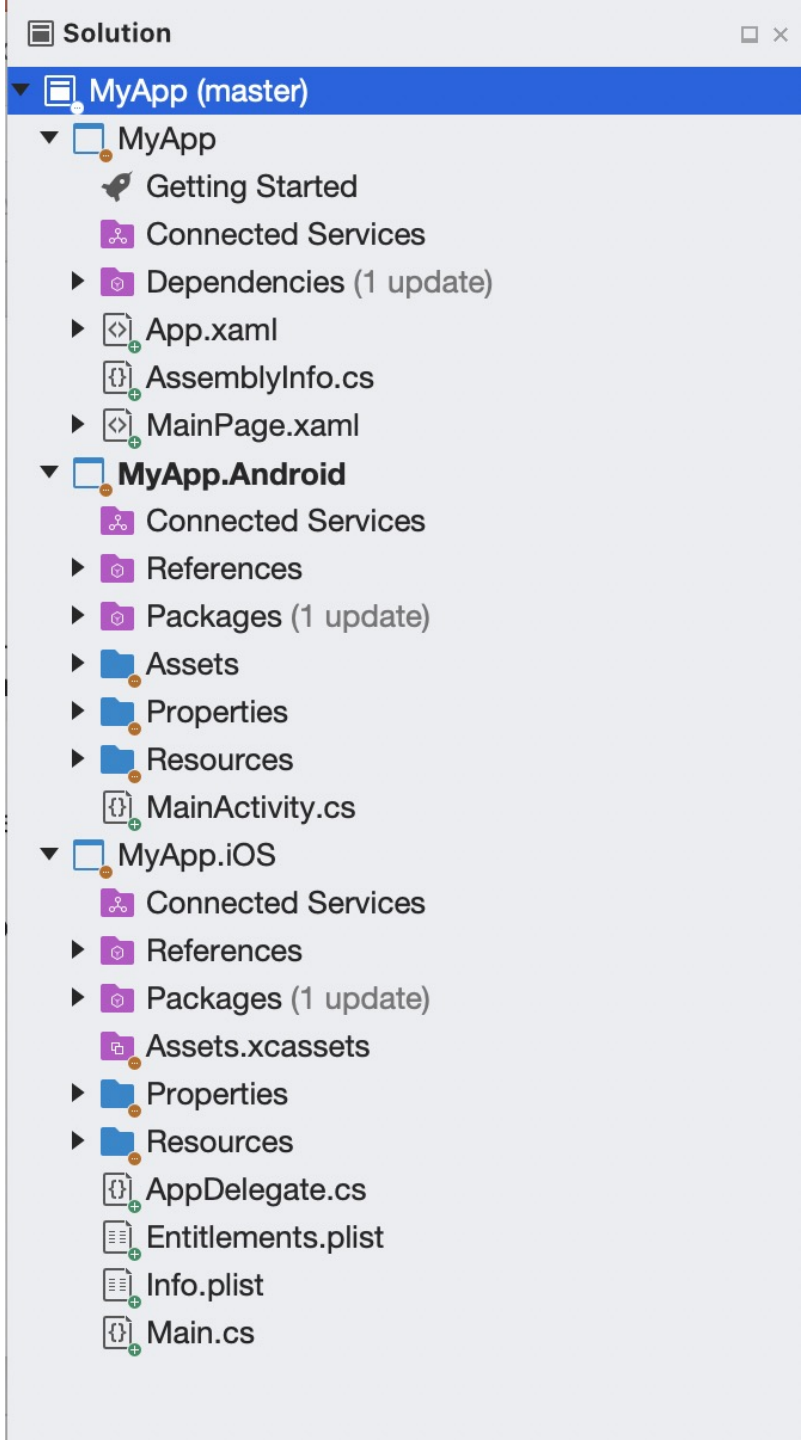
```
0 references
public ICommand ClickCommand { get; }
1 reference
public string Text { get; set; } = "Click me";

int count = 0;
0 references
void ExecuteClickCommand()
{
    count++;
    Text = $"You clicked {count} times.";
}
```

Single Project

Single project alleviates several points of friction when developing apps that target multiple platforms.

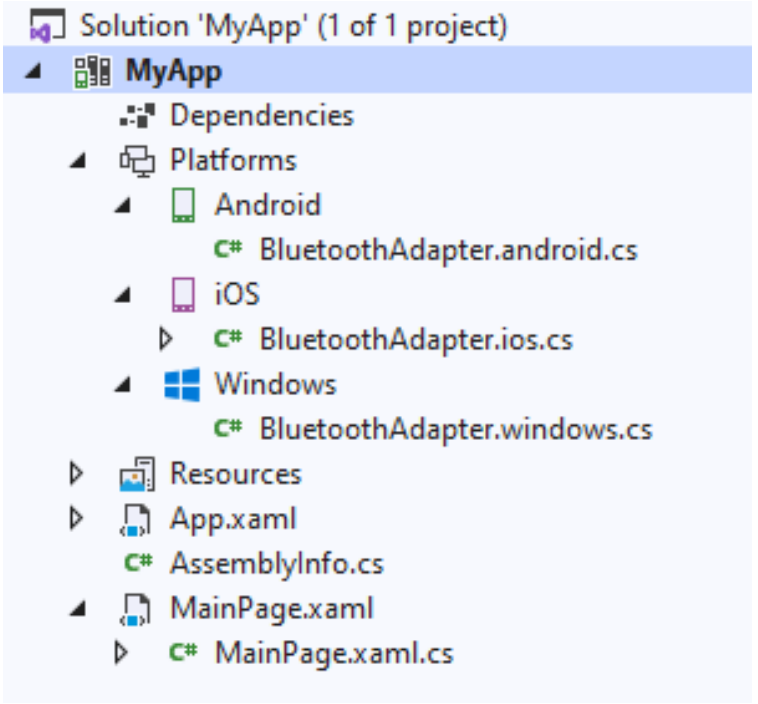
- Images
- Fonts
- Platform code



Single Project

Single project alleviates several points of friction when developing apps that target multiple platforms.

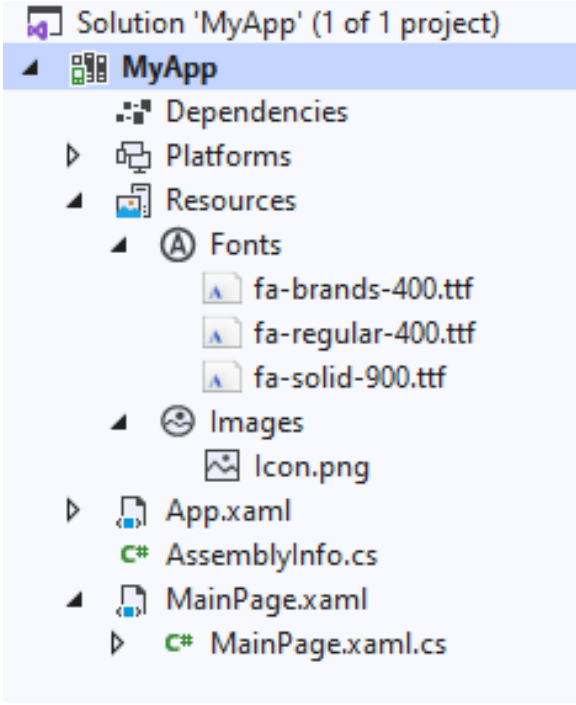
- Multi-targeting
- Images
- Fonts
- Platform code



Single Project

Single project alleviates several points of friction when developing apps that target multiple platforms.

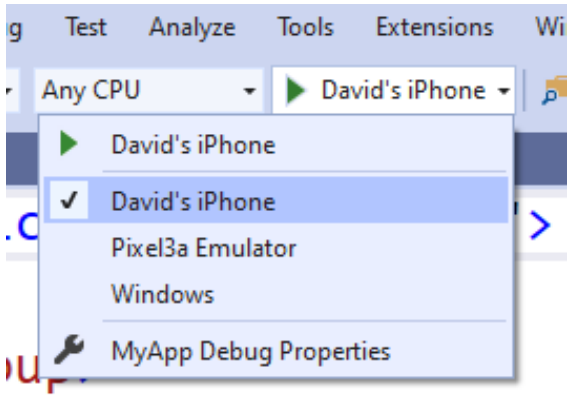
- Images
- Fonts
- Platform code



Single Project

Single project alleviates several points of friction when developing apps that target multiple platforms.

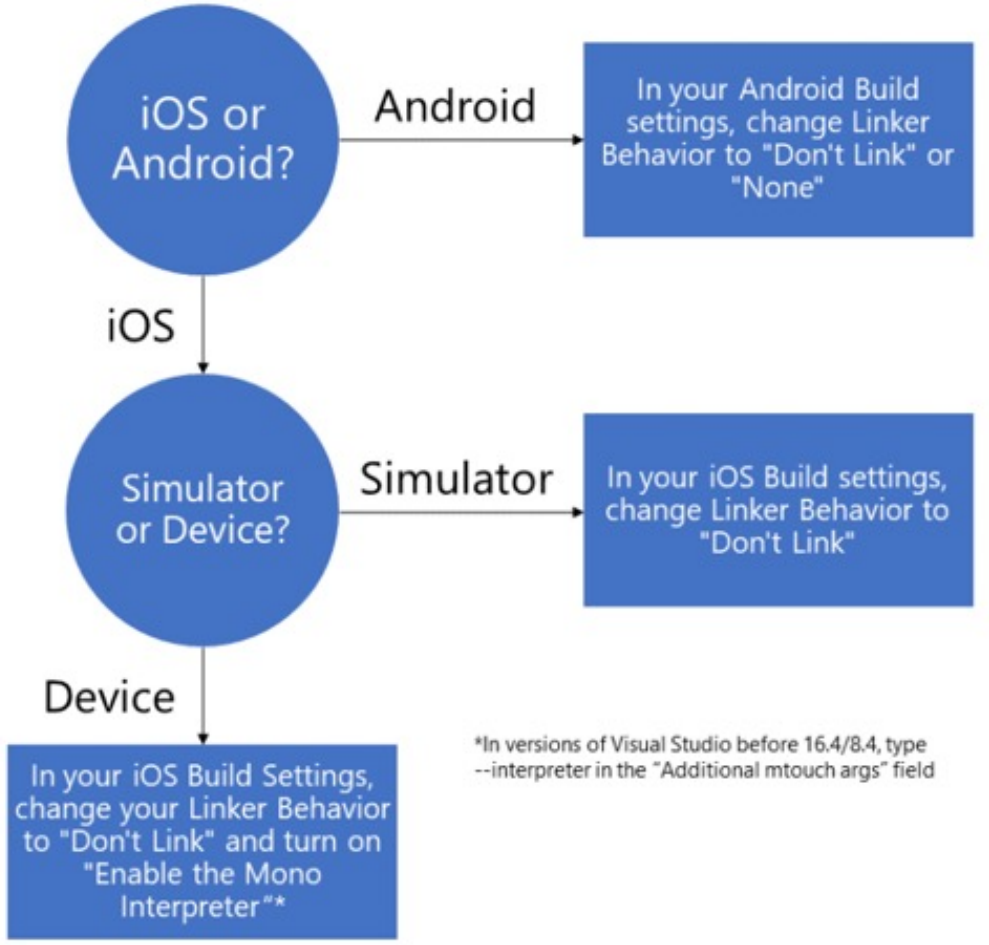
- Images
- Fonts
- Platform code



Hot Reload

Iterate on your application without time consuming rebuilds.

- Android, iOS, macOS, and Windows
- XAML and C#
- State retention
- Multi-devices
- No debugger required



Hot Restart

Extends Hot Reload experience to resources and references.

- iOS from Windows
- Android from macOS or Windows

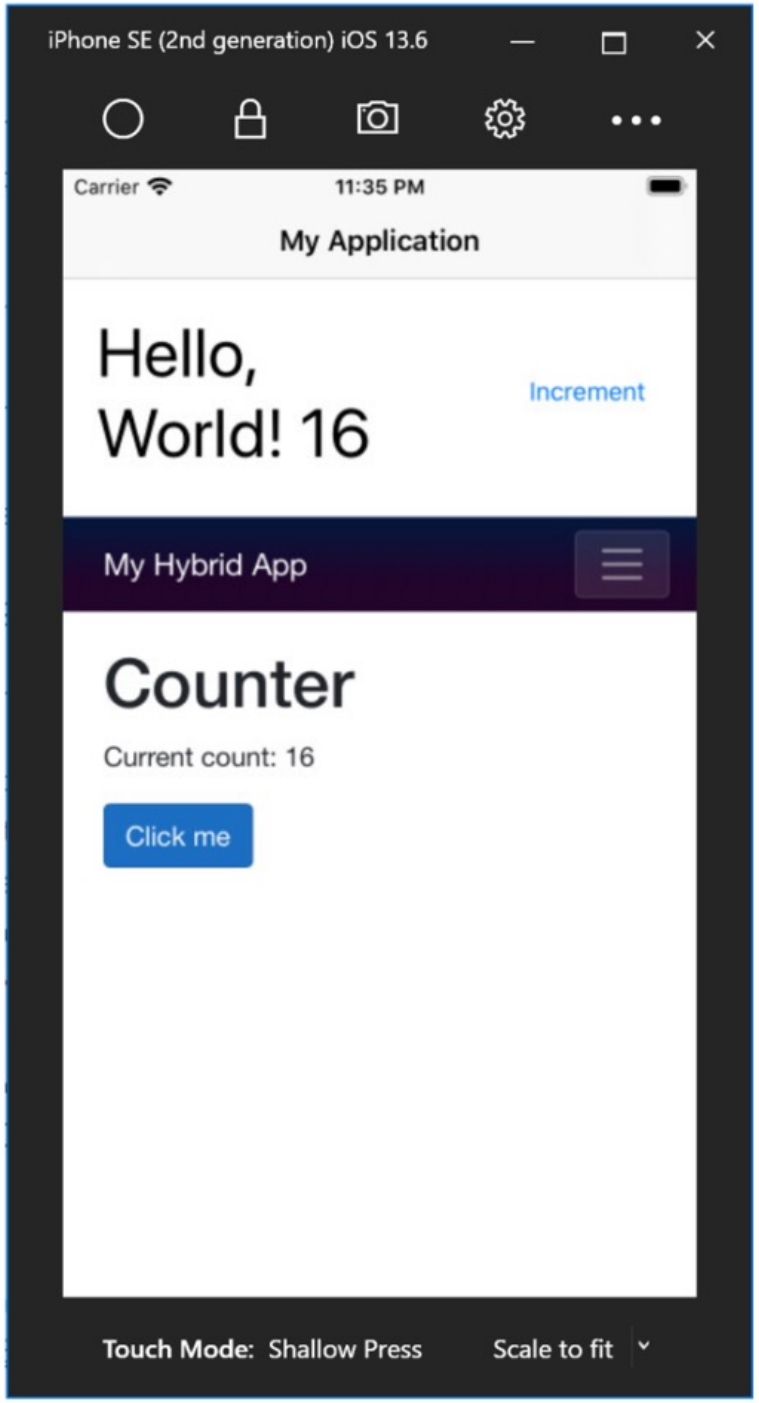


Experiments

What experiences can we build on top of these performant, native APIs?

Blazor

- Razor + .NET MAUI controls
- Hybrid mode
- Visual Studio Code
- Debugging

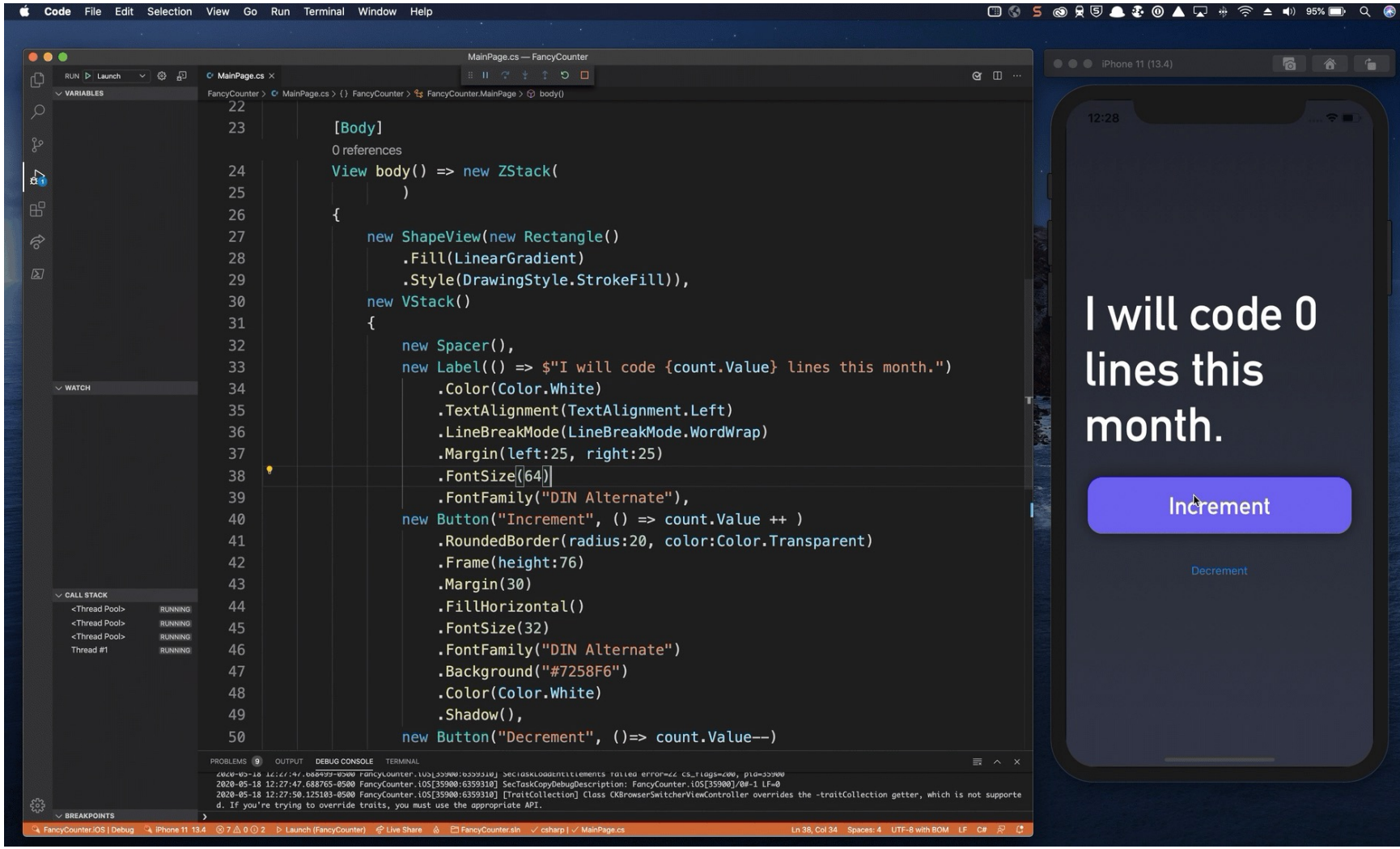


Native UI

Web UI

Comet

- Unidirectional flow of state
- C# fluent syntax
- Focused hot reload
- Visual Studio Code
- Debugging



FAQs

Questions, worries, and the hunt for the cheese.

Should I use Xamarin.Forms now, or wait for .NET MAUI?

Xamarin.Forms. Your project will upgrade to .NET MAUI.

Will you migrate my solution to “Single Project”?

No, your existing projects will continue to work without Single Project. We will migrate projects in place to the .NET 6 common project system.

Documentation will be provided if you wish to manually adopt Single Project.

Do I need to rewrite custom renderers?

No. We are making sure renderers still work.

You will want to migrate them at your convenience.

Will 3rd party UI libraries continue to work?

Yes! At a minimum they will need to recompile adding a target for .NET 6 and switch namespaces.

Will .NET MAUI run on Linux?

Linux is not a supported platform in this version of .NET MAUI.
There is an unofficial backend implemented with GTK.

What “flavor” of XAML will .NET MAUI use?

The same XAML you use in Xamarin.Forms today.

We will revisit some naming changes later in the preview.

Where can I follow progress?

github.com/dotnet/maui

Refreshed regularly from xamarin/xamarin.forms

Thanks for joining!

